Computer Networks

Exam Study Guide

A concise, diagram-led revision of the full syllabus — data link layer, error control and correction, network layer, IP, routing, addressing, transmission, LAN/MAC protocols, IEEE 802, multiplexing and spread spectrum.

How to use this guide. It follows the layer-by-layer logic of the course and groups the 15 lecture decks into 9 study sections plus a formula sheet. Each section opens with a banner, keeps the exam-critical definitions in bold, and isolates every formula in an amber line so it is easy to memorise the night before.

Section	Topic	Source decks
1	Data Link Layer fundamentals, framing, services	1Data Link Layer
2	Error detection, ARQ, parity, CRC, checksum	2Error Control
3	Error correction: Hamming, convolutional, Reed-Solomon, LDPC	3Error Correction
4	Network layer, IP datagram, fragmentation	1Network Layer, 2Internet Protocol
5	Routing algorithms and protocols	3Routing Algorithms
6	IP addressing, classful & classless, subnetting	4IP Addressing
7	Digital & analog transmission, modulation, PCM	5Digital, 6Analog, 8Spread spectrum
8	Data link protocols: stop-and-wait, sliding window, ARQ	4Data Link Layer Protocols
9	SDLC/HDLC, LAN MAC, IEEE 802, multiplexing	5SDLC&HDLC;, 6LANMAC, 7IEEE802, 7Multiplexing

1. Data Link Layer — Fundamentals

Role and position

• Physical layer offers only a **raw bit-stream** service; the DLL turns it into a **reliable link** and controls activation, maintenance and deactivation of the link.

• Responsible for the **error-free transfer of data frames**, defines the data format on the network, and uniquely identifies each device on a LAN.

• Lowest layer that performs **hop-by-hop** (node-to-node) delivery; the network layer above does the end-to-end job.

IEEE 802 splits the DLL into two sublayers

Sublayer	Position	Responsibility
LLC — Logical Link Control	Upper	Flow control, error correction, resequencing for connection-oriented service; also supports connectionless. Hands packets up to the network layer and identifies the network-layer protocol from the header.
MAC — Media Access Control	Lower	A method for stations to gain access to the shared medium; the bridge between LLC and the physical layer; transfers packets over the network.

Functions of the DLL

■ **Framing** — group raw physical-layer bits into frames.

■ **Physical addressing** — identify the sender/receiver on the link.

■ **Error control** — detect and recover from transmission errors (CRC, ARQ).

■ **Flow control** — keep a fast sender from drowning a slow receiver.

■ **Access control** — decide which device controls the link at any instant.

■ Provides a well-defined **service interface** to the network layer, plus general link management.

Services offered to the network layer

Service	Guarantees	Where it fits
Unacknowledged connectionless	No ACK; losses recovered at higher layers.	Reliable media (coax, fibre) with low error rate; suitable for voice where delay matters more than occasional bad data.
Acknowledged connectionless	Each frame acknowledged.	Unreliable media (wireless). ACKs add delay; useful as an optimisation, since resending a small frame beats resending a whole packet at the network layer.
Acknowledged connection-oriented	Most reliable: every frame arrives, exactly once, in order.	Guaranteed service; needs explicit setup/teardown and sequencing.

End-to-end vs hop-by-hop

*In the **hop-by-hop** model each link acknowledges the previous node; in the **end-to-end** model only the final destination acknowledges. DLL reliability is hop-by-hop.*

Framing techniques

Framing decides how the bit stream is chopped into frames and how the receiver re-synchronises after an error.

Method	Idea	Weakness / note
Character (byte) count	A header field gives the number of bytes in the frame, so the receiver knows where it ends.	If the count is garbled by one bit flip the receiver loses sync; even with a checksum it cannot find the next frame start. **No longer used.**
Flag bytes with byte stuffing	Each frame begins and ends with the same special **flag byte**; a data byte equal to the flag is preceded by an escape byte (stuffing).	Assumes a fixed 8-bit character, so it cannot handle heterogeneous environments.

Flag bytes with bit stuffing	Flag pattern 01111110; transmitter inserts a 0 after every five consecutive 1s of data; receiver removes it after five 1s.	Works for arbitrary bit-oriented data; the basis of HDLC/SDLC.
Physical-layer coding violations	Reserved (redundant) signals that never occur in normal data delimit frames.	Only possible when the physical encoding has spare symbols.

Byte stuffing example

Data: A ESC B FLAG C → transmitted: A ESC ESC B ESC FLAG C (escape added before ESC and FLAG).



General frame shape: flags delimit, FCS detects errors.

2. Error Detection, ARQ, Parity, CRC & Checksum

Error control basics

• Reliable delivery needs **positive and negative feedback** (ACK/NAK) plus **timers**. A timer rescues the case where a frame vanishes and neither ACK nor NAK arrives.

• Retransmission can deliver a frame more than once → give frames **sequence numbers**.

• Use **error detection** on reliable media (retransmission is cheap); use **error correction** on unreliable media (retransmission may fail again).

• Errors are handled at the **data link** or **transport** layer. Two strategies: add enough redundancy to deduce the original (error-correcting codes, a.k.a. **Forward Error Correction**), or add just enough to notice an error (error-detecting codes) and request retransmission.

Error types

Type	Description	Notes
Single-bit	Exactly one bit flips.	Least likely in serial transmission — noise would need duration ~ one bit time (e.g. 1 µs at 1 Mbps). Common in parallel transmission.
Multiple-bit	Two or more (non-consecutive) bits flip.	Occurs in both serial and parallel links.
Burst	A run of bits from the first to the last corrupted bit changes.	Most likely in serial links, because noise usually lasts longer than one bit. Length = distance from first to last corrupted bit; inner bits may be unharmed.

Burst length depends on data rate

At 1 kbps, noise of 1/100 s → 10 bits affected; at 1 Mbps the same noise → 10,000 bits.

Ternary view: detect vs correct

• **Detection** only asks whether any error happened; a single-bit error is as bad as a burst.

• **Correction** needs to know the exact number of corrupted bits **and their location** — harder, so the message size and error count matter.

• Correction methods: **FEC** (receiver guesses the message from redundant bits; best when errors are few) or **retransmission** (receiver asks for the message again).

• **Redundancy** is the central idea: extra bits added by the sender, removed by the receiver. **Block coding** and **convolutional coding** are the two families; convolutional coding is more complex.

Modular arithmetic

Work in a limited integer range 0..N-1 with modulus N; a value ≥ N is reduced to its remainder. Modulo-2 arithmetic uses only {0,1}:

Addition: 0+0=0, 0+1=1, 1+0=1, 1+1=0 | Subtraction identical → both are XOR.

Block coding & Hamming distance

• Split the message into **k-bit data words**; add **r** redundant bits to make **n = k + r** bit **code words**. Encoding is one-to-one, so $2^n - 2^k$ code words go unused.

• A receiver detects a change only if it holds the valid code-word list **and** the received word is invalid.

• **Hamming distance** d(x,y) = number of differing bit positions = number of 1s in x ⊕ y.

• **Minimum Hamming distance** d_{min} = smallest distance over all pairs.

• A scheme is written **C(n, k)** with an expression for d_{min}.

• **Linear block code**: XOR of any two valid code words is again valid; d_{min} = number of 1s in the lightest nonzero code word.

Detect up to s errors → $d_{min} \geq s + 1$

Error-detecting codes (all linear, systematic, block)

Code	How it works	Strength
Parity (VRC)	One extra bit makes the count of 1s even (or odd); equivalently XOR of data bits.	$d_{min} = 2$, detects all single-bit errors; a long burst is caught with only probability ½.
Two-dimensional parity	Parity per row and per column; interleaving spreads a burst across columns.	Isolates and can even correct a single bit; catches most bursts.
Longitudinal (LRC)	Arrange data in rows/columns, send parity of each column.	Detects burst and single-bit errors; fails only for 2-bit errors in the same vertical slice.
Checksum	Arithmetic sum of message words (one's complement), negated and appended.	Detects all odd-error cases and most even ones; weakest for compensating bit pairs.

Cyclic Redundancy Check (CRC)

• Treat a block of data as coefficients of a polynomial and divide by a fixed **generator polynomial G(x)**; both high- and low-order bits of G(x) are 1.

• The transmitter generates an **n-bit frame check sequence (FCS)** so the whole (k + n)-bit frame is exactly divisible by G(x). The receiver divides again; a nonzero remainder means an error.

CRC algorithm (modulo-2 division)

■ Let r = degree of G(x). Append r zero bits to the frame (now m + r bits, polynomial $x^rM(x)$).

■ Divide the $x^rM(x)$ bit string by G(x) using modulo-2 division.

■ Subtract the r-bit remainder (modulo-2) from $x^rM(x)$; the result is the transmitted frame.

Checksum (incl. Internet 16-bit checksum)

• The checksum is a group of check bits; placed at the end of the message as the **complement of the sum**. Adding the whole received codeword should give zero.

• Sender: initialise 0 → add all data items (checksum counts as one) → **wrap** the carry (add surplus bits back) → complement → send.

• Receiver: add all items including checksum → wrap → complement; result 0 means clean, otherwise drop the packet.

• **Internet checksum** uses 16-bit words and one's-complement addition; the complement of the sum is the checksum.

Sender: checksum = complement(wrapped one's-complement sum). Receiver accepts iff recomputed value = 0.

3. Error Correction Codes

Two ways to correct

• Ask the sender to **retransmit** the whole data unit, or

• Use an **error-correcting code** that automatically fixes certain errors.

• A frame carries **m data bits + r redundant (check) bits**. A **linear** code computes the r check bits as a linear (XOR) function of the m data bits; a **systematic** code sends the m data bits unchanged alongside the check bits; a **block** code computes r solely from the m bits of the same block.

• Four families of error-correcting codes: **Hamming**, **binary convolutional**, **Reed-Solomon**, **low-density parity-check (LDPC)**. All add redundancy.

How many redundancy bits? (single-bit correction)

To correct a single error and identify which bit, m data + r check bits must satisfy:

$$2^r \geq m + r + 1 \quad \text{(total message sent = m + r)}$$

Hamming codes

• Number codeword bits from 1 leftward; positions that are **powers of 2** (1, 2, 4, 8, 16, …) are **check bits**; the rest carry data bits.

• Each check bit forces the parity of some collection of bits (including itself) to be even (or odd).

• A data bit at position k is checked by exactly the check bits appearing in the power-of-2 expansion of k — e.g. 11 = 1 + 2 + 8, so bit 11 is checked by bits 1, 2, 8; 29 = 1 + 4 + 8 + 16.

• The **(11, 7)** Hamming code has 7 data bits and 4 check bits (m = 7, r = 4, $2^4 = 16 \geq 12$).

• Performing the checks yields a nonzero **error syndrome** that pinpoints the bad bit — e.g. check results 0,1,0,1 give syndrome 0101 = 4 + 1 = 5, so bit 5 is wrong.

Worked check-bit example (even parity)

Check	Data positions covered	Count of ones	To make parity even
R1	3, 5, 7, 9, 11	3	R1 = 1
R2	3, 6, 7, 10, 11	4	R2 = 0
R4	5, 6, 7	2	R4 = 0
R8	9, 10, 11	1	R8 = 1

Using Hamming distance to correct errors

A code list with Hamming distance $d_{min} = 2d + 1$ can correct d single-bit errors. Example legal code words 0000000000, 0000011111, 1111100000, 1111111111 have HD 5 → can correct 2 single-bit errors. Interleaving Hamming codes also corrects burst errors.

Convolutional codes

• Not a block code: the encoder processes a **stream** of input bits and generates a stream of output bits — there is no natural message boundary.

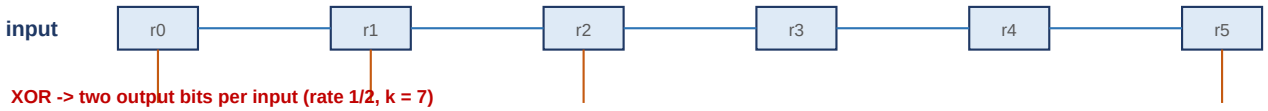
• Output depends on current **and previous** input bits, so the encoder has **memory**; the number of previous bits it depends on is the **constraint length k**.

• Specified by **rate** and **constraint length**; used in GSM, satellite and 802.11.

• The **NASA** code (Voyager, 1977; reused in 802.11) has **rate 1/2, k = 7**: one input bit produces two XOR-sum output bits — binary, linear, **not systematic**.

• Six memory registers; bits shift right each step. Input 111 from all-zero state gives states 100000, 110000, 111000 and outputs 11, 10, 01; seven shifts flush an input, hence k = 7.

• Decoded by the **Viterbi** algorithm: walk the observed sequence and, for each step and possible internal state, keep the input sequence producing fewest errors; the survivor with fewest errors is the most likely message.



NASA rate-1/2, k = 7 convolutional encoder (schematic).

Reed-Solomon codes

• Linear, often systematic, block codes that operate on **m-bit symbols** (not single bits).

• Based on the fact that an n-degree polynomial is fixed by n + 1 points: send data points plus check points on the same curve; if one arrives in error, fit the curve to recover it.

• For m-bit symbols, codewords are 2^m – 1 symbols; with m = 8 (bytes) a codeword is 255 bytes. The **(255, 233)** code adds 32 redundant symbols to 233 data symbols.

• With **2t** redundant symbols, up to **t** symbol errors are corrected. A single-bit error and an m-bit burst both count as one symbol error.

• Decoded by the Berlekamp-Massey algorithm; used in DSL, cable, satellite, CD/DVD/Blu-ray. Often nested inside a convolutional code to mop up bursts.

LDPC codes

• Linear block codes (Gallagher, 1962) where each output bit depends on only a **fraction** of the input bits, so the parity-check matrix has a **low density of 1s**.

• Decoded by an iterative approximation that converges on the best-fit legal codeword.

• Practical for large blocks; excellent error-correction that beats many alternatives, so it is rapidly entering new protocols.

4. Network Layer, Internet Protocol & Fragmentation

What the network layer does

• Gets **packets from source all the way to destination**, possibly across many hops through intermediate routers — the lowest layer that handles **end-to-end** transmission.

• **Routing / path determination**: choose the route packets take (routing algorithms).

• **Inter-networking**: the same network layer spans differently-physical networks; headers are stripped and the packet is passed between dissimilar links.

• **Congestion control**: when incoming rate at a router exceeds outgoing rate. Causes include many inputs competing for one output, infinite memory (queued packets time out and their duplicates add load), slow CPUs and low-bandwidth lines.

• **Switching**: move a packet from the router input to the correct output.

• **Call setup**: some architectures set up a path before data flows.

Design issues & services to the transport layer

Service should be …	Meaning
Independent of router technology	The transport layer must not depend on how routers are built.
Shielded from router details	Hide the number, type and topology of routers.
Uniform numbering plan	Addresses must be consistent across LANs and WANs.

Store-and-forward packet switching

A host sends a packet to its nearest router (on its LAN or over a point-to-point link). The router stores the packet until it has fully arrived (so the checksum can be verified), then forwards it toward the destination. Data is split into small packets transmitted node to node, processed and forwarded — **store-and-forward switching**.

Connectionless (datagram) vs connection-oriented (virtual circuit)

Aspect	Datagram subnet (connectionless)	Virtual-circuit subnet (connection-oriented)
Setup	None; each packet routed independently.	A path is established before data; a route is chosen and stored in router tables at setup.
Packet addressing	Each packet carries the full destination address.	Each packet carries a short virtual-circuit identifier.
Ordering	Packets may take different paths and arrive out of order.	Arrive in order along the reserved path.
Release	—	Releasing the connection terminates the circuit.
Trade-off	Address parsing time and per-packet overhead.	Setup time and resources, but reserves resources up front for QoS and congestion avoidance.

Routers

On each arriving packet a router strips the frame header/trailer, passes the payload to routing software, which uses an internal table (per destination) to pick an output line. The algorithm that builds and maintains these tables is the **routing algorithm**.

Internet Protocol (IP)

• One of the major TCP/IP protocols, working at the **network layer (OSI)** / **Internet layer (TCP/IP)**.

• Provides a **connectionless (datagram) service**: each packet treated separately. Packets at this layer are **datagrams** — header + data.

• Header is **20–60 bytes**, customarily drawn in **4-byte** rows. IP gives **best-effort delivery**: no guarantee, but it does its best.

• Common to all routers; identifies hosts by logical addresses (IPv4 = 32-bit) and routes data among them.

IP design issues

Issue	Key points
Routing	Static tables (may hold alternates) or dynamic tables updated for congestion/errors; **source routing** (source lists trusted routers) and **route recording** (routers append their address, for tracing/debugging).
Datagram lifetime	**TTL** field prevents indefinite looping: hop count (decrement per router) or time count (needs a global clock); on expiry the datagram is discarded, not forwarded.
Fragmentation / reassembly	Networks have different max packet sizes; reassembly in IP is **at the destination only**.
Error control	Delivery not guaranteed; a router may notify the source of a drop (TTL expiry, congestion, bad checksum). Notifications are not guaranteed.
Flow control	Limited in connectionless systems — e.g. the ICMP **source quench** packet asks for reduced flow.
Addressing	Each end and intermediate system gets a network-level address (IP address; OSI calls it NSAP). Each process gets a **port** (SAP in OSI).

IPv4 datagram header fields



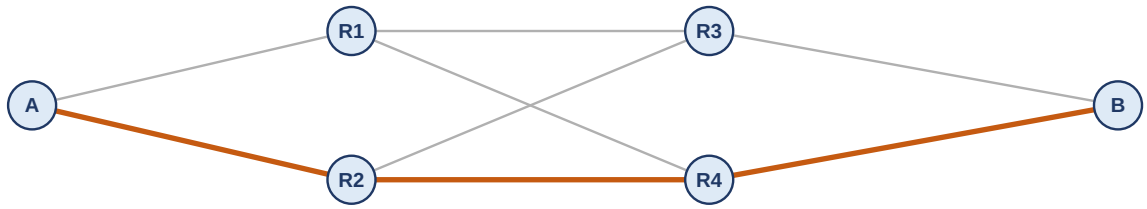
Rows are 32 bits; IHL counts 32-bit words, minimum 5 (= 20 octets).

Field	Meaning
Version	IP version used (IPv4 = 4).
IHL	Internet header length in 32-bit words, including options; min 5 (20 octets).
DSCP / ECN	Differentiated services code point (was Type of Service) for QoS; ECN signals congestion seen on the route.
Total Length	Whole IP packet (header + payload) in octets; max datagram = 65,535.
Identification	16-bit value that is identical across all fragments of one original packet, so the receiver can group them.
Flags	More-fragments (1 if more follow) and Don't-Fragment.
Fragment Offset	Position of the fragment's data in the original datagram, in multiples of 8 octets (64 bits).
TTL	Hops the packet may still cross; decremented each hop, discarded at 0.
Protocol	Next higher-layer protocol (ICMP = 1, TCP = 6, UDP = 17).
Header Checksum	Checksum of the header only; verified and recomputed at each router.
Source / Destination Address	32-bit sender / receiver addresses.
Options & Padding	Present when IHL > 5 (Security, Record Route, Timestamp, …); padding brings the header to a multiple of 32 bits.

Data field: any octet length, bounded by the 65,535-octet datagram limit.

Datagram vs virtual-circuit forwarding (concept)

Datagram: packets may split across routes. VC: one reserved path (orange toward B).



IP fragmentation and reassembly

• A packet larger than a link's **Maximum Transmission Unit (MTU)** must be fragmented or discarded (with an ICMP error to the source). Fragments are smaller than the smallest MTU on the path.

• Two approaches: **IP router segmentation** (routers fragment) or **IP Path MTU Discovery** (the sender fragments). Router fragmentation offloads work onto routers and can cause cascaded, odd fragmentation — **not recommended** in the modern Internet; Path MTU Discovery is recommended.

• Reassembly happens only at the destination. Fragments with the same 16-bit **Identification** are stored in one buffer at the offset given by the **fragment offset**; incomplete packets wait until all fragments arrive or a timer expires, then complete packets go up a layer.

• The sender places a fresh random value in the Identification field per packet; if a router fragments, every fragment copies the original value.

• Offset values let the receiver order fragments **even if they arrive out of order** or are re-fragmented; the **More-Fragments** bit is 0 on the last fragment only.



Three fragments of one original packet: same ID, increasing offset, MF=1 for all but the last.

*Reassembly failure is handled with a **reassembly lifetime** timer on the first fragment; if it expires, partial data is discarded to free buffers.*

5. Routing Algorithms

Forwarding vs routing

• Two processes live inside a router: **forwarding** handles each arriving packet by looking up its outgoing line in the routing table; **routing** fills in and updates those tables — that is where the routing algorithm works.

Desirable properties

Property	Meaning
Correctness	Packets actually reach their proper destination.
Simplicity	Low overhead; complexity raises overhead.
Robustness	Copes with topology/traffic changes and router crashes without a reboot.
Stability	Reaches equilibrium and stays there under all conditions.
Fairness	Every node gets a fair chance (often first-come-first-served).
Optimality	Optimal throughput and minimum mean packet delay. Fairness and optimality often conflict.

Classification

Adaptive (dynamic)	Non-adaptive (static)
Based on current traffic/topology measurements. Sub-types: **centralized** (one node holds all info — single point of failure), **isolated** (node decides alone), **distributed** (info from neighbours).	Route computed off-line in advance and loaded at boot. Includes **flooding**, **random walk**, and static **shortest-path** routing.

Adaptive sub-types in detail

• **Centralized**: one central node collects the whole topology/traffic picture and distributes it; only one node needs the information, but its failure downs the network.

• **Isolated**: the node decides without outside information. Example **hot potato** — drop the packet on the shortest output queue regardless of where it leads; a variant combines static link weights with queue lengths.

• **Backward learning**: routing tables adapt from incoming packets carrying source identity + a hop counter. A node records the hop count seen; it updates only if the new value is better. **Weakness**: when the best route fails it cannot recall the second best, so nodes must periodically forget and restart.

• **Delta routing**: hybrid of centralized and isolated. Nodes send line costs to a central node, which computes the k best i→j paths. Path n is treated as equivalent to the best if $C_{ijn} - C_{ij1} <$ delta. As delta → 0 this becomes centralized; as delta → ∞ all paths become equivalent.

• **Distributed**: node uses information from neighbours; risk is that the network changes between receiving info and sending the packet.

Static strategies

• **Flooding**: every incoming packet is sent on every outgoing line except the one it arrived on. Robust (good for military use) but wastes bandwidth and can loop. Loop controls: **sequence numbers** (discard already-seen source+seq), **hop count** (drop at max), **spanning tree** (send only toward the destination); a practical variant is **selective flooding** (send only on roughly-correct-direction lines).

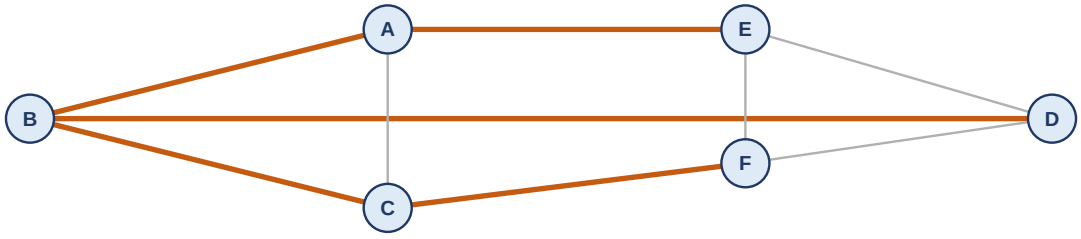
• **Random walk**: send to a randomly chosen neighbour; highly robust and makes good use of alternative routes when the network is highly connected; usually implemented by choosing the least-queued link.

• **Multipath / bifurcated routing**: each node keeps a table row per destination listing best, second-best, … outgoing lines with relative weights; the packet picks one at random using the weights as probabilities. Tables are computed manually and fixed.

The optimality principle and sink trees

If router J lies on the optimal path from I to K, then the optimal path from J to K follows the same route. Consequently, the set of optimal routes from all sources to one destination forms a tree rooted at the destination — a **sink tree**. Every routing algorithm ultimately aims to discover and use sink trees.

A subnet (grey) and its sink tree rooted at B (orange): the unique optimal path to B from every node.



Shortest-path routing

• Model the subnet as a graph: routers = nodes, links = arcs, each arc labelled with a **cost** (lower is better). Find the shortest path on the graph.

• Cost metrics: **minimum hops** (unit cost, breadth-first search — ignores load), **transmission + propagation delay** (reflects capacity and distance but is static), **queuing delay** (reflects load but not propagation), or a blend of distance, bandwidth, traffic, communication cost, mean queue length and measured delay.

• Assumptions: nodes cost zero w.r.t. themselves, links symmetric ($d_{i,j} = d_{j,i}$), graph complete (missing edge = infinite cost).

Bellman-Ford (distance-vector style, iterates on hop count)

D[i, h] = shortest path length from i to node 1 using at most h hops.

Init: $D[i,0] = \infty$ for $i \neq 1$, $D[1,h] = 0$. Iterate: $D[i,h+1] = \min_j \{ d_{i,j} + D[j,h] \}$. Stop when $D[i,h] = D[i,h+1]$ for all i.

*Uses the optimality principle; terminates because at most N–1 hops are needed. Complexity **$O(N^3)$**.*

Dijkstra (link-state style, iterates on path length)

Sets P = nodes with known shortest distance; Q = the rest. Initially P = {1}. Repeatedly move the minimum-cost node of Q into P, then relax:

$$D_j = \min[\, D_j,\ D_i + d_{j,i} \,] \quad \text{(i just moved to P)}$$

*Each moved node's shortest path is then final; N–1 iterations; complexity **$O(N^2)$**.*

Floyd-Warshall (all-pairs, iterates on allowed intermediates)

$$\text{Init } D_{i,j}[0] = d_{i,j}. \quad D_{i,j}[n+1] = \min\{ D_{i,j}[n],\ D_{i,n+1}[n] + D_{n+1,j}[n] \}.$$

*Grow the intermediate set from {1} to all nodes; gives all-pairs shortest paths. Complexity **$O(N^3)$**.*

Distance-vector routing

• Dynamic algorithm: each router keeps a vector of best known distance to every destination and the line to use; tables are updated by exchanging information with neighbours.

• With delay as metric and known neighbour delays, every T ms each router sends its estimated delay list to each neighbour and receives theirs.

• Example: router J measures delays to A, I, H, K of 8, 10, 12, 6 ms. To reach G, if A claims 18 ms, J can reach G via A in 26 ms; via I 41; via H 18; via K 37 — so J picks H with 18 ms. Repeat for all destinations.

• **Count-to-infinity problem**: DV reacts fast to good news but slowly to bad news, because when X tells Y it has a path, Y cannot tell whether it itself is on that path.

Link-state routing

Replaced distance-vector routing, whose problems were (1) ignoring line bandwidth (delay metric = queue length) and (2) slow convergence (count-to-infinity). Each router:

■ Discovers its neighbours and learns their network addresses.

■ Measures the delay/cost to each neighbour.

■ Builds a packet telling everything it has learned.

■ Sends that packet to all other routers (link-state packets, with a buffer).

■ Computes the shortest path to every other router (e.g. Dijkstra).

Because every link appears twice (once per direction), a full set of link-state packets lets a router rebuild the entire subnet graph.

Other routing topics

• **Hierarchical routing**: group routers into regions to shrink tables; a router knows full detail inside its region but only a summary of others.

• **Broadcast routing**: **reverse path forwarding** — a router forwards a broadcast only if it arrived on the link that is on the router's own shortest path back to the source; builds the broadcast tree implicitly from the sink tree.

• **Multicast routing**: one spanning/multicast tree per group, so a packet reaches only group members.

6. IP Addressing — Classful, Classless & Subnetting

Addressing modes

Mode	Destination	Note
Unicast	One host (32-bit destination = that host).	Client → one server.
Broadcast	All hosts on the segment; address 255.255.255.255.	Every host must process it.
Multicast	A group; addresses start 224.x.x.x.	Entertained by more than one host.
Anycast	Nearest host offering a service.	Connection goes to the first responder — best connection.

Every network reserves one address for the network number and one for broadcast.

Interpreting a 32-bit address

10000000 00111101 00010111 11011000 → 128.61.23.216. Writing it with a /24 says the first 24 bits are network, last 8 are host (a classless address). IPv4 uses a **hierarchical** scheme: network → sub-network → host.

Classful addressing

Class	Leading bits	Range	Net / host split	Networks	Hosts per net	Mask
A	0	1.0.0.0 – 126.255.255.255	8 / 24	126 (2^7–2)	16,777,214 (2^{24}–2)	255.0.0.0
B	10	128.0.0.0 – 191.255.255.255	16 / 16	16,384 (2^{14})	65,534 (2^{16}–2)	255.255.0.0
C	110	192.0.0.0 – 223.255.255.255	24 / 8	2,097,152 (2^{21})	254 (2^8–2)	255.255.255.0
D	1110	224.0.0.0 – 239.255.255.255	—	multicast	—	none
E	1111	240.0.0.0 – 255.255.255.255	—	experimental	—	none

*127.x.x.x is reserved for **loopback**; 0.0.0.0 means "this computer" (bootstrap). Masks are 32-bit numbers: contiguous 1s (network) then 0s (host); ANDing an address with its mask yields the network address.*

Special address forms

Prefix (network)	Suffix (host)	Meaning
all 0s	all 0s	this computer (used during bootstrap)
network address	all 0s	identifies the network
network address	all 1s	broadcast on the specified network
all 1s	all 1s	broadcast on the local network
127	any	loopback (testing)

Why classful failed

• Each class has a fixed number of fixed-size blocks; a class A block is far too large for almost any organisation, class B often too large, class C too small — so addresses were wasted.

• Class D was sized for 268,435,456 multicast groups (never needed) and class E went largely unused.

• Rapid Internet growth then caused **address depletion**. Remedies: **subnetting** (split a large block) and **supernetting** (combine class C blocks). Drawback: routing becomes more difficult.

Subnetting, supernetting and masks

• **Subnetting**: divide a class A/B block into contiguous subnets and assign each to a smaller network. It **increases the number of 1s** in the mask (e.g. /24 → /26). Each LAN gets a subnet number; the host portion splits into subnet + host. A site still looks like one network to the outside.

• **Supernetting**: combine several contiguous class C blocks into a supernet; it **decreases the number of 1s** (e.g. four class Cs: /24 → /22).

• **CIDR** (Classless Inter-Domain Routing): use addresses independent of class; slashed notation /n (n = 8, 16, 24 …). Used on addresses advertised to the Internet (ISP side).

• **VLSM** (Variable-Length Subnet Mask): different masks for different subnets — "subnets of subnets" — used inside companies.

Classless addressing

• No classes, but addresses are still granted in **blocks** whose size fits the entity (a household may get 2, a large ISP hundreds of thousands). Extends the subnet idea to the whole Internet and guards against exhaustion.

• Three restrictions on a block: addresses must be **contiguous**; the count must be a **power of 2**; the **first address must be divisible by the block size**.

• A block is defined as x.y.z.t/n. From the address and /n you can derive everything.

First address: set the 32–n rightmost bits to 0. Last address: set the 32–n rightmost bits to 1. Number of addresses = 2^{32-n}.

Worked example — 128.42.5.4/21 (netmask 255.255.248.0)

Quantity	Method	Result
Prefix	Count contiguous 1s in the netmask.	/21 (twenty-one 1s)
Network address	AND address with mask.	128.42.0.0
Broadcast address	Force all host bits to 1 (OR with inverted mask).	128.42.7.255
Host count	$2^{32-21} - 2$	2046

CIDR notation example: 128.140.168.0/21 → mask 255.255.248.0, lowest host 128.140.168.1, highest host 128.140.175.254.

Subnetting calculations you may be asked for

• **Prefix for at least H hosts**: host bits = ceil($\log_2$ H); prefix = 32 – host bits. (H = 100 needs 7 host bits → /25.)

• **Longest mask containing two addresses**: convert both to binary and find the longest common leading bit string (e.g. 128.42.5.17 and 128.42.5.67 → /25).

• **Subnet example**: 205.16.37.39/28 → first 205.16.37.32, last 205.16.37.47, 16 addresses (2^{32-28}).

• First address = address AND mask; last address = address OR (complement of mask); number of addresses = complement of mask + 1.

Maximum hosts per subnet = $2^{32 - \text{netmask_length}} - 2$ (subtract all-zeros network and all-ones broadcast).

7. Digital & Analog Transmission, Modulation & Spread Spectrum

Analog-to-digital conversion (PCM)

• Analog data (voice, video) is a continuous waveform; to send it over digital signals we convert it. Two techniques: **Pulse Code Modulation (PCM)** and **delta modulation**.

• PCM has three steps: **sampling** (take samples of x(t) at uniform intervals), **quantization** (map each sample to a finite-precision value), and **binary encoding/compression**. Before sampling, filter to limit the maximum frequency (removing high-frequency components that distort the shape).

• Sampling interval T_s; sampling rate $f_s = 1/T_s$.

Nyquist: $f_s \geq 2 \times$ (highest frequency in the signal).

*Sampling methods: **ideal** (impulse at each instant — not easily implemented), **natural** (short pulse retaining the analog shape), and **flat-top / sample-and-hold** (the common method, constant amplitude per sample).*

Quantization

• Divide the min–max amplitude range into **L zones** of height Δ = (max − min)/L; assign each zone the value of its **midpoint** (0 … L–1).

• Bits per sample: $n_b = \log_2 L$.

• **Quantization error** = difference between the actual and coded (midpoint) value; more zones → smaller Δ → smaller error but higher bit rate.

• Lower-amplitude signals suffer more (error range $\Delta/2$ is fixed), so use **non-linear quantization** to keep the signal-to-quantization-noise ratio fixed: logarithmic levels, or **companding** (compress at sender, expand at receiver, fixed-height zones).

Example: Vmin = –20 V, Vmax = +20 V, L = 8 → Δ = 40/8 = 5; zones –20..–15, …, +15..+20 with midpoints –17.5, –12.5, … 17.5; n_b = 3 bits, codes 000–111.

Bit rate = $n_b \times f_s$. (A digitized signal always needs more bandwidth than the original analog signal.)

PCM decoder: a hold circuit keeps each pulse's amplitude until the next arrives, then a low-pass filter with cutoff = the highest pre-sampled frequency recovers the signal. Larger L gives less distortion.

Delta modulation & variants

• Send only the **difference** between successive samples: 1 if the pulse rose, 0 if it fell. Works well for small changes; large changes cause large errors.

• **Delta PCM (DPCM)**: use more bits to quantize the difference — more bits, more levels, higher accuracy.

Transmission modes

Mode	How	Cost / note
Parallel	n wires send n bits per clock tick.	n× faster than serial but needs n lines — expensive, so limited to short distances.
Serial — asynchronous	Byte-wise with start bit (0) and stop bit(s) (1), gaps allowed.	No shared clock at byte level; timing unimportant, patterns used instead.
Serial — synchronous	Frames of many bytes, no gaps between bytes.	Faster (no extra bits/gaps); receiver must separate bits — for high-speed links.
Serial — isochronous	Guarantees a fixed data rate with equal gaps.	Needed for real-time audio/video where uneven frame delays are unacceptable.

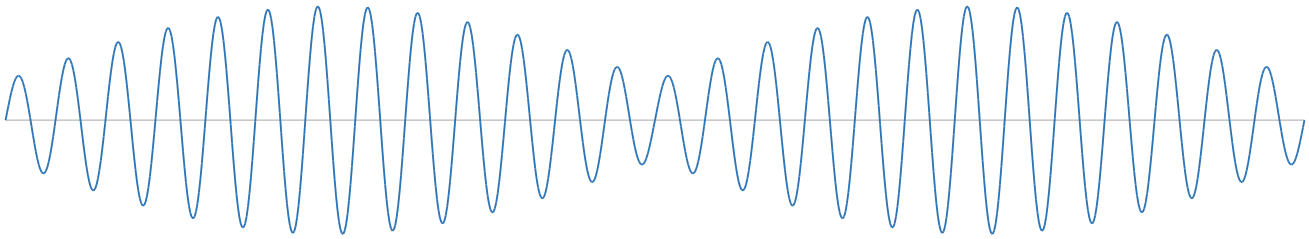
Digital-to-analog conversion (modulation)

Carry digital data on an analog **carrier** by altering one of its three characteristics: amplitude, frequency or phase. Mechanisms: ASK, FSK, PSK, and **QAM** (amplitude + phase together — the most efficient, used today).

Bit rate N and baud rate S: S = N × (1/r) bauds, where r = bits per signal element. Signal rate S ≤ bit rate N.

Scheme	Idea	Bandwidth
ASK (OOK)	Change carrier amplitude; e.g. 0 = no signal, 1 = signal.	$B = (1+d)S$, $d \in [0,1]$
FSK	Change carrier frequency: f1 = fc + Δf (1), f2 = fc – Δf (0).	$B = (1+d)S + 2\Delta f$
Multi-level FSK	Several frequencies, L levels.	$B = (1+d)S + (L-1)/2\Delta f = L \cdot S$
PSK	Change carrier phase; robust to amplitude noise.	$B = (1+d)S$
QPSK	Two carriers in quadrature (90°); 2 bits per symbol, L = 4.	More bits per symbol → higher bit rate
QAM	Combine ASK + PSK; constellation of amplitude/phase points.	Most efficient

*Constellation diagrams plot in-phase carrier (X) vs quadrature carrier (Y): ASK/BPSK lie on the X axis; QPSK uses four points (phases 45°, 135°, –135°, –45°, amplitude 2√2); QAM fills a grid. **Coherent** FSK switches at the same phase; **non-coherent** FSK does not.*



Amplitude-modulated carrier: the modulating signal is the envelope; the carrier frequency and phase stay constant.

Analog-to-analog conversion

• Represent analog information by an analog signal; needed when the medium/available channel is **band-pass** (e.g. radio: each station's low-pass signal is shifted to a different band).

Scheme	What changes	Bandwidth
AM	Amplitude only; frequency & phase fixed; modulating signal is the envelope.	B_{AM} = 2B (one sideband may be dropped).
FM	Frequency follows the modulating voltage; amplitude & phase fixed.	$B_{FM} = 2(1 + \beta)B$, β usually 4 (≈10× signal frequency).
PM	Phase follows the modulating voltage; amplitude & frequency fixed.	$B_{PM} = 2(1 + \beta)B$, $\beta \approx 2$. Bandwidth higher than AM.

Spread spectrum

• Trade bandwidth efficiency for robustness: transmit each signal over a bandwidth $\mathbf{B_{ss}} \blacksquare \mathbf{B}$, much wider than needed. B_{SS} is much larger than B, giving redundancy and a protective envelope.

• The spreading is done **after** the signal is created, using a spreading code (pseudo-random sequence) independent of the original signal.

• Goals: immunity to narrowband interference, noise and multipath; jamming resistance (AJ); low probability of intercept (LPI / hiding); encryption; and multiple users sharing the same band.

• Types: **FHSS** (rapidly hop the carrier among sub-bands in a code-controlled order — interference affects only a short interval), **DSSS** (each data bit represented by several bits via XOR with the spreading code, widening the spectrum), plus **THSS** (time hopping) and **CSS** (chirp: wideband linear frequency-modulated pulses). IEEE 802.11 uses FHSS or DSSS.

8. Data Link Protocols: Flow Control, Stop-and-Wait & ARQ

• **Flow control** stops the sender overwhelming the receiver (buffer overflow). **Transmission time** = time to emit all bits; **propagation time** = time for a bit to traverse the link.

• Protocols split by channel: **noiseless** vs **noisy**.

Protocol	Assumptions	Behaviour
Unrestricted simplex (Utopia)	Infinite receiver buffer, error-free, sender always ready, one direction.	Framing only — no error or flow control. Hypothetical.
Stop-and-wait (noiseless)	Finite buffer & processing speed; still error-free.	Sender sends one frame and waits for the receiver to signal it is ready — flow control only. Good for a few large frames.
Stop-and-wait ARQ (noisy)	Finite buffer; errors/losses possible.	Positive ACK + retransmission: sender keeps a copy, waits a finite time for ACK; on timeout or NAK, retransmits; on ACK, sends the next frame.

ARQ building blocks

• Positive ACK; a **1-bit sequence number** in frames (not in ACKs); a **timeout** to detect lost frames/ACKs; retransmission. Can fail under early timeout.

• Full-duplex works with **piggybacking**: temporarily delay an ACK so it can ride on the next outgoing data frame.

• **Fragmentation**: split large blocks into small frames to bound buffer size, detect errors sooner, retransmit less on error, and stop one station hogging the medium. (Plain stop-and-wait then becomes inadequate.)

Sliding-window flow control

• Allow multiple frames in transit: the receiver has a buffer of W frames; the sender may send up to W frames before an ACK.

• Each frame is numbered; the ACK carries the number of the **next frame expected**. Sequence numbers are bounded by the field size k and wrap modulo 2^k.

• Sender window grows when an ACK arrives; receiver window is constant and advances when the low-edge sequence number arrives; frames outside the window are discarded.

• The three sliding-window protocols: **one-bit** (stop-and-wait + error control, piggybacked ACKs), **Go-Back-N**, and **Selective Repeat**.

Goal	Window	ACK style	On error
Go-Back-N	Send up to N frames; **cumulative** ACK (ACK N if all < N received).	Cumulative; out-of-order packets discarded, no receiver buffering.	Retransmit all frames from the earliest unacked one — wastes bandwidth if error rate is high.
Selective Repeat	Retransmit only the erroneous/lost frame; good frames buffered.	Not cumulative (individual ACKs); out-of-order arrival handled.	Only the bad frame resent on NAK; loses time if frames are lost in bursts.

Selective Repeat max window = (Sequence Number Space)/2 when out-of-order buffering is used; = Sequence Number Space − 1 otherwise.

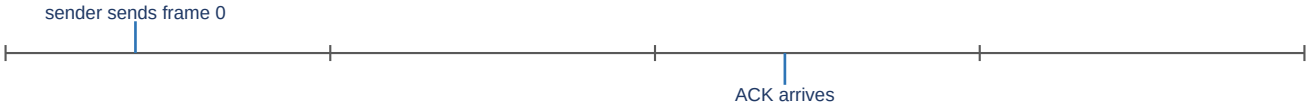
Sliding-window enhancements & timers

• Receiver can send **Receive-Not-Ready** to stop flow without a data ACK, then a normal ACK to resume.

• Piggybacking with an **auxiliary ACK timer**: set the timer when a packet needing an ACK arrives; if no reverse data appears, generate an ACK when it fires.

• **Negative ACK (NAK)** on an out-of-order frame triggers retransmission of that frame; NAKs are not cumulative and duplicate NAKs are not sent.

• Selective Repeat may use N timers or a single timer simulated in software (queued timeouts).



Pipelining: with a receiver window of 1 a single error forces retransmission of the whole window (Go-Back-N); a large receiver window lets good frames be buffered (selective repeat).

9. SDLC/HDLC, LAN MAC, IEEE 802 & Multiplexing

SDLC & HDLC

• **SDLC** (IBM): synchronous bit-oriented protocol; simplex/half/full duplex; sliding window; EBCDIC; 256-character data frame; a single control field does all link control; **master–slave** (primary/secondary) only.

• Primary issues commands; secondary returns responses. States: transient (before/after transmission and line turnaround), idle (15+ consecutive 1s), active (transmitting info/control).

• General frame: Flag(8) | Address(8) | Control(8) | Data (multiple of 8) | FCS(16) | Flag(8).

• **HDLC** generalises this: primary/secondary/combined station types; unbalanced (one primary, many secondaries) or balanced (two combined) link configurations.

• Frame types: **I-frame** (user data, with piggybacked flow/error control), **S-frame** (supervisory: RR, RNR, REJ, SREJ), **U-frame** (unnumbered: link management, mode setting, connect/disconnect — SNRM, SARM, SABM, DISC, UA, DM, RD, UP, RSET, XID, TEST, FRMR).

• Flag = 01111110; **bit stuffing** inserts a 0 after every five 1s. Address field identifies the secondary (8 bits, extensible in multiples of 7; all-ones = broadcast). **P/F bit**: P in command frames (poll), F in response frames (final). FCS = 16-bit CRC (optional 32-bit).

• Modes: **NRM** (secondary transmits only when polled), **ARM** (secondary may initiate; handles own timeouts/retransmission), **ABM** (identical stations, either may initiate). Operation phases: initialization, data transfer, disconnect.

• Channel states: **active** (transmitting frame/abort/time-fill), **idle** (15+ consecutive 1s — primary must repoll), **abort** (7+ contiguous 1s).

IEEE 802 LAN architecture

• Corresponds to the lower two OSI layers; most LANs follow **IEEE 802**. Layer 2 is split into **LLC** (interface to higher levels, flow control, based on classical DLC) and **MAC** (framing, error detection, address recognition, access to medium).

• Topologies: **bus**, **ring**, **star**. MAC coordinates access to the shared broadcast medium so simultaneous transmissions do not collide.

Multiple-access taxonomy & random access

Protocol	Rule	Max throughput
Pure ALOHA	Transmit any time; after a frame wait up to $2 \cdot t_{prop}$ for an ACK, else wait a random time and resend. Vulnerable time = 2T.	$S = G \cdot e^{-2G}$, S_{max} = 0.184 at G = ½ (≈18%)
Slotted ALOHA	Time in slots = one frame time; transmit only at a slot start; a central clock marks slots. Vulnerable time = T.	$S = G \cdot e^{-G}$, S_{max} = 0.368 at G = 1 (37%)
Reservation ALOHA	Group N slots; each station owns one slot; unused slots reusable.	TDM-like, adapts to load



ALOHA advantages: a lone node can use full rate R; simple; no master. Disadvantages: collisions when M nodes transmit, giving each far below R/M and low utilisation.

Carrier-sense multiple access (CSMA)

• Sense the medium before transmitting (**listen before talk**); propagation time is small, so collisions happen only when stations start within the propagation window. Vulnerable time = maximum propagation time.

• **Non-persistent**: if idle transmit; if busy wait a random time and retry — reduces collisions but wastes idle bandwidth during backoff.

• **1-persistent**: if busy, keep listening and transmit immediately when idle — avoids idle time but two ready stations collide (selfish).

• **p-persistent**: for slotted channels — if idle send with probability p, defer to next slot with 1–p; larger p → smaller utilisation and less waiting.

CSMA/CD, CSMA/CA and controlled access

• **CSMA/CD** (Ethernet, bus topology): listen while talking. On detecting a collision the sender aborts and sends a 48-bit **jam signal**, backs off a random time, then retransmits. Detect a collision in the worst case in **2× maximum propagation delay**. Frame must be long enough to detect the collision, else CSMA/CD is pointless.

• **Binary exponential backoff**: after the k-th collision pick $R \in [0, 2^k - 1]$ and wait R × slot time (Ethernet slot = 51.2 µs at 10 Mbps); cap k at 10 (max 1023); give up after 16 attempts. Gives a last-in-first-out effect among waiting stations.

• **CSMA/CA** (IEEE 802.11 WLAN): collision **detection is not used** because a node cannot listen and transmit at once. Distributed Coordination Function (DCF): sense the channel; if free for a DIFS transmit; else defer and use a random backoff in [0, W–1], decrementing only while idle and freezing when busy; ACK sent after a SIFS; if no ACK, retransmit after EIFS (doubling CW to a maximum).

• **Hidden terminal**: A cannot hear C's transmission to B and wrongly starts, causing collision at B. **Exposed station**: B hears A's transmission and wrongly refrains from sending to C. RTS/CTS optionally mitigates these.

• Random-access performance: simple, decentralised, low delay at low load; but limited throughput, unbounded delay under heavy load, and potentially unfair.

Controlled access	Mechanism
Reservation	Cycles begin with a reservation interval of N minislots (one per station); a station reserves in its own minislot; everyone learns the order; reserved stations then send at full rate.
Polling	Centralized (primary polls secondaries via select/poll frames; NAK if nothing; round-robin or predetermined order) or distributed (known polling order, highest priority gets access then passes it on).

Token passing	A token authorises a node to transmit; no master; station interface has a listen state (copy if addressed, else forward) and a transmit state (capture free token, send frames, reinsert token, remove own frame).

Channelization: share bandwidth by frequency (FDMA), time (TDMA) or code (CDMA). In FDMA/TDMA a node is limited to R/M even when alone; in CDMA all transmissions share one channel, each coded so receivers can separate them.

IEEE 802.3 (Ethernet)

• CSMA/CD with slotted channels: if idle transmit; if busy wait for an idle slot then transmit. Stations listen while transmitting; on collision they cease and wait a random (exponentially backed-off) time.

• Contention detail: no ACKs, so a station must ensure all stations see its transmission and no collision occurred — it transmits for the worst-case round-trip time (for 2500 m of coax at 10 Mbps, ≈ 50 µs).

• Minimum frame: 50 µs at 10 Mbps → 500 bits, rounded up to **512 bits (64 bytes)**; maximum frame 1518 bytes. (RFC 894 encapsulates IP in Ethernet; RFC 1042 in IEEE 802.)

• Frame: Preamble (alternating 1/0 for clock sync) | SFD 10101011 | Destination MAC | Source MAC | Length (LLC data) | Data | FCS (32-bit CRC, excludes preamble and SFD). Addresses are IEEE-assigned.

• 48-bit MAC address burned into the NIC; first 3 bytes assigned by IEEE. Least-significant bit of the first byte: 0 = unicast, 1 = multicast; broadcast is FF:FF:FF:FF:FF:FF.

• 10 Mbps variants: 10Base2/10Base5 (bus coax), 10BaseT with hub (one collision domain, star, twisted pair), 10BaseT with switch (one collision domain per port). Half-duplex send-xor-receive; full duplex when attached to a switch port. Manchester encoding.

• Speeds: Fast Ethernet 802.3u (100 Mbps; 100BASE-TX/FX/T4, TX most popular), Gigabit 802.3z/ab (1000 Mbps, enterprise backbone), 10 Gigabit 802.3ae (10 Gbps, 64b/66b line code).

• Performance: $U = T_{trans} / (T_{trans} + T_{prop})$ with no collisions; add $T_{contention}$ when collisions occur.

Token bus (802.4) & token ring (802.5)

• **Token Bus**: token ring over a **virtual ring** on a bus/tree coax (industrial use). Each station knows its left/right neighbours; a logical ring is formed in address order (coordinator initialises). Token passes high→low address; holder transmits for a limited time then hands over; priorities via a queue and token-holding-time fractions. Frame: Preamble | Start Delimiter | Frame Control | Dest Addr | Source Addr | Payload | Checksum | End Delimiter. Messages: token T(ID, nextID) and data M(ID). Ring maintenance handles a station leaving (tell predecessor the new successor) and joining (open a window; conflict resolved by sending a request whose length encodes the last 2 ID bits). High token overhead (512 bit times).

• **Token Ring** (802.5, IBM): ring topology, unidirectional, **Differential Manchester** encoding, token passing, 4/16 Mbps. Frame: SD | AC | FC | DA | SA | PDU | CRC | ED | FS. AC bits: P (priority), T (token), M (monitor), R (reservation). Status bits A (address recognised) and C (frame copied). A station converts a free token to a data frame and back when its frame returns. **Active monitor** (chosen by CLAIM_TOKEN election) acts as master clock, inserts a 24-bit delay, ensures exactly one token, detects a broken ring and removes circulating frames; standby monitors take over.

Multiplexing & switching

• **Multiplexing** shares a link between sessions; unlike MAC (geographically separate users, so contention), multiplexing is centralised in a switch, so coordination is easier. Static multiplexing gives each user a fixed portion; dynamic (statistical) multiplexing varies by demand.

• **FDM**: divide the spectrum into disjoint bands, each on a different carrier with **guard bands** (e.g. broadcast radio); supports full duplex; high initial cost; carrier must match precisely.

• **WDM**: FDM for optical fibre — combine light sources (LED/laser) at different wavelengths; 40–128 channels at 2.5–10 Gbps each; can be all-optical; often combined with TDM.

• **TDM**: divide time into slots in cyclical order (synchronous) — preassigned fixed slots, wasted if idle; data rate is n× faster and unit duration n× shorter; **interleaving** takes bits from each input. **Statistical TDM** allocates slots on demand, so the line rate can be below the aggregate of inputs.

• **Switching**: how traffic moves input→output at nodes. **Circuit switching** establishes a dedicated path (establish, transfer, disconnect) — inefficient for data (idle resources, fixed rate, setup time). **Message switching** stores the whole message. **Packet switching** splits messages into numbered packets (~1000 octets) with routing info, buffered at each node (store-and-forward); circuit switching pairs with static mux, packet switching with statistical mux.

10. Formula & Quick-Reference Sheet

Error control

Hamming: $2^r \geq m + r + 1$ | detect s errors $\rightarrow d_{min} \geq s+1$ | correct d errors $\rightarrow d_{min} = 2d+1$

Flow / link efficiency

Transmission time = frame bits / bit rate. Propagation time = distance / speed.

Transmission & modulation

Nyquist: $f_s \geq 2f_{max}$. Quantization $\Delta = (max-min)/L$, $n_b = \log_2 L$. PCM bit rate = $n_b \times f_s$.
S = N/r. ASK/PSK: B = (1+d)S. FSK: B = (1+d)S + 2Δf. Multi-level FSK: B = L·S.
AM: B = 2B. FM: B = 2(1+β)B. PM: B = 2(1+β)B.

Multiple access

Pure ALOHA: $S = G \cdot e^{-2G}$, $S_{max} = 0.184$ at G = ½. Slotted ALOHA: $S = G \cdot e^{-G}$, $S_{max} = 0.368$ at G = 1.
CSMA/CD worst-case collision detection = 2 × max propagation delay. Ethernet slot = 51.2 µs; retry $R \in [0, 2^k-1]$; give up after 16.

IP addressing

Block of /n: first = addr AND mask, last = addr OR ¬mask, count = 2^{32-n}, hosts = $2^{32-n} - 2$.
Prefix for H hosts: host bits = ■$\log_2 H$■, prefix = 32 − host bits.

Routing complexity

Bellman-Ford $O(N^3)$ | Dijkstra $O(N^2)$ | Floyd-Warshall $O(N^3)$

Rapid-fire revision list

Topic	Remember
DLL services	Unacknowledged connless / acked connless / acked conn-oriented
Framing	Byte count (dead), byte stuffing, bit stuffing, coding violations
Error detect	Parity (d=2), CRC (divisible by G(x)), checksum
Error correct	Hamming $2^r \geq m+r+1$, convolutional (K=7, rate ½), RS, LDPC
IP header	IHL=5→20 B; TTL; protocol 1/6/17; ID+offset+MF for fragments
Fragmentation	Reassemble only at destination; Path MTU Discovery preferred
Routing	Sink tree; Bellman-Ford, Dijkstra, Floyd-Warshall; DV vs link-state
Addressing	Classes A–E, masks, subnet/supernet, CIDR, VLSM
Modulation	ASK/FSK/PSK/QAM; AM/FM/PM; PCM; delta; spread spectrum
LAN MAC	ALOHA 18%/37%, CSMA variants, CSMA/CD, CSMA/CA, token passing
802.x	802.3 Ethernet, 802.4 token bus, 802.5 token ring, 802.11 WLAN

Exam tip: nearly every topic reduces to one of three questions — *where in the stack?*, *what goes wrong?*, and *which formula sizes it?* Anchor each answer to a layer, a failure mode and a number.